

Digitale Bürgerabstimmungsplattform

Zwischenpräsentation

Projektteam Team 6 (E-Vote)

Fabian Meintzer, Fabian Scholl, Heike Fritz, Mareike Starker, Jacqueline Schulz

Master Medieninformatik (online)

Moderne Softwareentwicklung

Prof. Dr. S. Ipek-Ugay • 24. November 2025

Projektthema & Kontext

Problemstellung

Digitalisierung demokratischer Prozesse für transparente und sichere Bürgerabstimmungen

Zielsetzung

Entwicklung einer benutzerfreundlichen, sicheren Abstimmungsplattform mit hoher Datenintegrität

Technologie-Stack

Python, PyCharm, GitHub Actions, SonarCloud, pytest

Event Storming / Domain Storytelling

Fachliche Anforderung (User Stories):

- Bürger kann sich registrieren
- Bürger kann einmal abstimmen
- Bürger kann Abstimmungsübersicht / Ergebnisliste ansehen

>>> Begriffe und Fachlogik bilden fachliche Grundlage für Domain-Driven-Design

Bounded Contexts & Aggregates

Bürgerverwaltung (BC)

Aggregat: Bürgerverwaltung

Aggregat Root:

Buerger (Entities / Value Objects)

Methoden: editiereDaten()

Domain Services:

→ BürgerValidieren

Application Services:

→ Registrieren

Technical Service:

→ Authentifizierung

Infrastruktur (json)

Event:
Bürger
stimmt
ab

Abstimmungsmanagement (BC)

Aggregat: Ergebnis

Aggregat Root: Ergebnis (Entities / Value Objects)

Methode: berechneErgebnis(), ergebnisDetails()

Aggregat: Abstimmung

Aggregat Root: Abstimmung

Value Object: Stimme

Methode: stimmeAbgeben(), anzahlStimmen()

Regel: Jeder Bürger eine Stimme

Domain Services / PolicyService:

→ Nur wahlberechtigte Bürger

→ QuotenService

Application Services (nutzt PolicyService):

→ AbstimmungsService

→ AbstimmungsübersichtService

Test-Driven Development Integration

TDD-Zyklus

1. **RED** - Test schreiben (schlägt fehl)
2. **GREEN** - Code implementieren (Test erfolgreich)
3. **REFACTOR** - Code optimieren

Was ist TDD?

Tests werden VOR der Implementierung geschrieben - Code entsteht als Antwort auf Tests

Warum TDD?

Höhere Codequalität durch sofortige Testbarkeit, Tests dokumentieren Anforderungen, weniger Bugs in Produktion

Nachteile von TDD

Tests müssen ebenfalls gepflegt werden, daher ist anfangs mehr Aufwand nötig; Integrationstests und Systemtests bleiben trotzdem unerlässlich.

TDD in der Praxis: Tests schreiben RED

Test-First Ansatz: Testfälle definieren vor der Implementierung

```
# Name Tests
@pytest.mark.parametrize("name, valid", [
    ("Anna-Maria", True),          # Happy Path
    ("Élodie", True),
    ("Maximilian", True),
    ("É", True),                  # Edge Case: sehr kurzer Name
    ("A" * 50, True),             # Edge Case: langer Name
    ("Anna--Maria", True),        # Edge Case: doppelte Bindestriche

    ("", False),                 # Negativ: leer
    ("Annal23", False),          # Negativ: Zahlen
    ("Anna@", False),           # Negativ: Sonderzeichen
    ("-Anna", False),           # Negativ: Bindestrich am Anfang
    ("Anna-", False),           # Negativ: Bindestrich am Ende
])
```

pytest.mark.parametrize: Mehrere Testfälle kompakt definiert (Happy Path, Edge Cases, Negative Tests)

TDD Implementation: Code schreiben GREEN

Implementierung mit Pydantic Validators für Domänenlogik:

```
@field_validator("name")
@classmethod
def validate_name(cls, name: str) -> str:
    """Validiert den Namen des Bürgers."""
    if not name or len(name) < 1:
        raise ValueError("Name ist zu kurz.")
    if not re.fullmatch(r"[A-Za-zÄ-ÿ\s-]+", name):
        raise ValueError("Name enthält ungültige Zeichen.")
    if name.startswith('-') or name.endswith('-'):
        raise ValueError("Name darf nicht mit Bindestrich beginnen oder enden.")
    return name
```

Pydantic: Bibliothek für Python zur automatischen Datenvalidierung

LLM in der Codegenerierung

Beispiel: Code-Ausschnitt aus "ergebnis.py"

```
class Ergebnis(BaseModel):
    ergebnisID: int
    abstimmungsID: int
    einzelwerte: List[Stimmenanzahl]
    timestamp: datetime = Field(default_factory=datetime.now)

    @field_validator("einzelwerte")
    @classmethod
    def validate_einzelwerte(cls, einzelwerte):
        if not einzelwerte or len(einzelwerte) == 0:
            raise ValueError("Es müssen Stimmen für mindestens eine Option vorhanden sein.")
        return einzelwerte
```

LLM half bei Edge Case Identifikation: Leere Listen-Validierung automatisch generiert

LLM als Pair-Programming Partner (1)

LLM-Vorschläge zur Code-Optimierung (Beispiel: ergebnis.py)

✗ Vorher: Redundante Methode

```
@property  
def getGesamtergebnis(self) -> int:  
    return self.gesamtergebnis
```

✓ Nachher: Optimiert

```
def gesamtergebnis(self) -> int:  
    return sum(e.anzahl for e in self.einzelwerte)
```

Die optimierte Lösung benutzt direkt ein Property `gesamtergebnis`, das die Summe aller Stimmen berechnet.

✗ Vorher:

Methode macht gar nichts Eigenes - ruft nur ein anderes Attribut (`gesamtergebnis`) auf und gibt es zurück

✓ Nachher:

optimierte Lösung nutzt Property `gesamtergebnis`, berechnet Summe aller Stimmen

Verbesserung:

kürzer, besser lesbar und folgt Python Empfehlung (statt künstliche Getter-Methoden wie `get...()`)

LLM als Pair-Programming Partner (2)

LLM-Vorschläge zur Code-Optimierung (Beispiel: authentifizierungs_service.py)

✗ Vorher: Naive Zeitstempel

```
if expires_delta:
    expire = datetime.utcnow() + expires_delta
else:
    expire = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

✓ Nachher: Zeitzone-sicher

```
now_utc = datetime.now(timezone.utc)
if expires_delta:
    expire = now_utc + expires_delta
else:
    expire = now_utc +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

✗ Vorher:

Vorherige Lösung verwendet `datetime.utcnow()`
-> liefert "naives" UTC-Datum ohne
Zeitzoneinformationen

✓ Nachher:

Saubere Zeitbehandlung:
optimierte Version nutzt
`datetime.now(timezone.utc)`
und erzeugt ein "timezone-aware"
UTC-Datum, wie von modernen
Python-Standards empfohlen

Korrekte Nutzung und Einbindung der LLMs

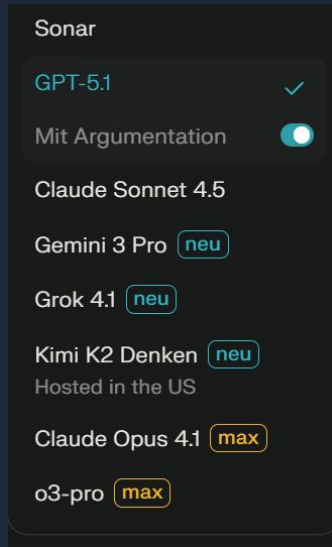
🔊 LLM-Auswahl & Konsistenz

Verhinderung qualitative Unterschiede im Code:
Einigung im Team auf Nutzung des neuesten Sprachmodells von OpenAI:
GPT 5.0, später 5.1

- ✓ Edge-Case-Identifikation (Tests)
- ✓ Code-Smell-Erkennung (Refactoring)
- ✓ Test-Generierung
- ✓

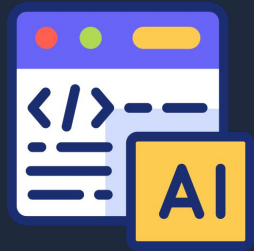
Best-Practice-Vorschläge

- LLMs als Pair-Programming-Partner wertvoll für Ideengeneration und Testabdeckung
- Entwickler dahinter muss bestehen bleiben: kritisch hinterfragen, testen, Kontext wahren
- Nur überprüfte und Tests-bestätigte Refactorings landen im Code



⚠️ Kritische Bewertung notwendig

- LLM-Vorschläge sind Ideengeneratoren, nicht Wahrheiten
- Jeder Vorschlag auf Fachkontext, Sicherheit und Tests geprüft
- Manche Vorschläge verworfen (z.B. zu aggressive Refactorings)
- Code-Quality-Tools (SonarQube) Validatoren für LLM-Ideen



SonarQube Cloud Integration

Qualitätssicherung

- Zur statischen Analyse und Bewertung der technischen Qualität von Quellcode
- Als Plugin in PyCharm
- SonarQube Cloud wurde mit dem GIT-Repository verbunden
- Integration in die GitHub Actions Pipeline
- Ergebnisse, Auswertungen und Verbesserungsvorschläge per Webinterface

SonarQube Cloud Integration

Qualitätssicherung

- Identifiziert Issues und ordnet diese Kategorien und Schweregraden zu, die jeweils gefiltert werden können

The screenshot displays the SonarQube Cloud 'Issues' page. On the left, a 'Filters' sidebar shows categories like 'Software quality' (1 issue), 'Security' (1 issue), 'Reliability' (4 issues), and 'Maintainability' (4 issues). Under 'Severity', there are 'Blocker' (1), 'High' (3), 'Medium' (4), and 'Low' (1). The main area shows a list of issues. The first issue is 'Delete this unreachable code or refactor the code to make it reachable.' with a 'Reliability' category and 'Medium' severity. The second issue is 'Rename this parameter "buergerID" to match the regular expression "[a-z][a-z0-9_]*\$' with a 'Maintainability' category and 'Low' severity. The third issue is 'Change this code to not reflect unsanitized user-controlled data.' with a 'Security' category and 'Blocker' severity. Each issue entry includes a description, category, severity, and a status of 'Open' and 'Not assigned'.

Summary Issues Security Hotspots More ▾

Filters

▼ Software quality

Security 1

Reliability 4

Maintainability 4

▼ Severity ⓘ

🔴 Blocker 1

🔴 High 3

🟡 Medium 4

🟢 Low 1

🔍 Info 0

> Code attribute

> Type

Select issues ▾ ▲ Navigate to issue ◀ ▶

7 issues 1h 2min effort

apps/.../domain/entities/ergebnis.py

Delete this unreachable code or refactor the code to make it reachable. Intentionality

Reliability 🟡 Medium cwe unused

☐ Open Not assigned L29 • 5min effort • 9 days ago • 🐛 Bug • 🚨 Major

apps/buergerverwaltung/api/v1/main.py

Rename this parameter "buergerID" to match the regular expression "[a-z][a-z0-9_]*\$". Consistency

Maintainability 🟢 Low convention

☐ Open Not assigned L53 • 2min effort • 8 days ago • 🐛 Code Smell • 🚨 Minor

Change this code to not reflect unsanitized user-controlled data. Intentionality

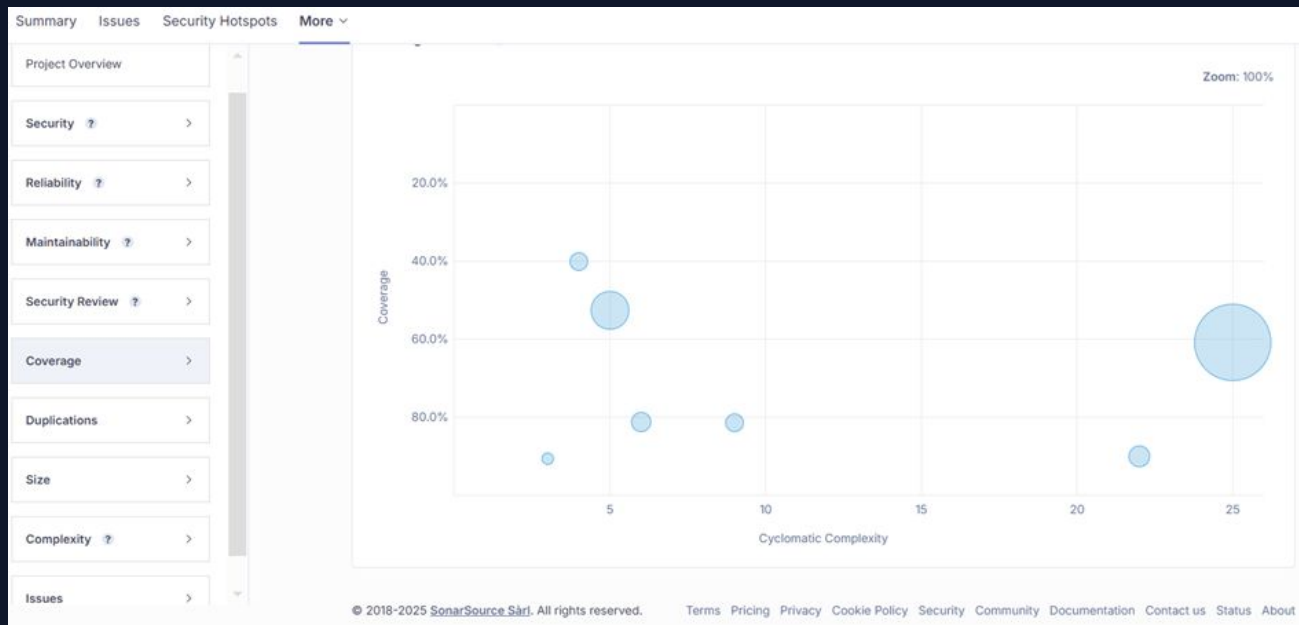
Security 🔴 Blocker cwe

☐ Open Not assigned L74 • 30min effort • 8 days ago • 🐛 Vulnerability • 🚨 Blocker

SonarQube Cloud Integration

Qualitätssicherung

- Analyse des Codes nach bestimmten Metriken wie Coverage und Security



SonarQube Cloud Integration

Analysebeispiel Coverage

- Empfohlener Wert der Testabdeckung: 80%
- Derzeitiger Stand (22.11.25)

Overall Code	
Coverage	72.2%
Lines to Cover	331
Uncovered Lines	92
Line Coverage	72.2%

- 331 Codezeilen sind abgedeckt/ 92 CZ sind nicht abgedeckt
- Wann ist es sinnvoll “technische Schuld” zu begleichen?

CI/CD: GitHub Actions

Einblick

- Integrierter Automatisierungsdienst von GitHub
- Führt Workflows bei Ereignissen aus (z. B. Push, Pull Request)
- Der CI-Workflow ist in der Datei `python-app.yml` definiert
- Unterstützt CI/CD: Tests, Code-Analyse, Qualitätssicherung
- Läuft in isolierten Umgebungen („Runner“ in der Cloud)



CI-Workflow für eVote



Checkout des Repositories & Python-Umgebung einrichten

- GitHub Actions nimmt den Quellcode aus dem Repository in die Build-Umgebung.
- `fetch-depth: 0` bedeutet: vollständige History, nicht nur der letzte Commit.
- Installiert / aktiviert Python 3.10 in der Pipeline.
- Vorteile: Fixe Version = reproduzierbare Builds

```
steps:
- name: Checkout
  uses: actions/checkout@v4
  with:
    fetch-depth: 0 # Vollständiger Checkout für SonarCloud

- name: Set up Python
  uses: actions/setup-python@0b93645e9fea7318ecaed2b359559ac225c90a2b
  with:
    python-version: "3.10"
```

CI-Workflow für eVote



PYTHONPATH & Test-spezifische Umgebungsvariablen

- Durch das Setzen von PYTHONPATH können die Module in den Tests sauber importiert werden
- TESTING=1 , kann im Code genutzt werden, um zu erkennen: „Ich laufe gerade in einer Test-/CI-Umgebung“

```
- name: Add repo to PYTHONPATH
  run: echo "PYTHONPATH=$GITHUB_WORKSPACE" >> $GITHUB_ENV

- name: Set test environment variables
  run: |
    echo "TESTING=1" >> $GITHUB_ENV
    echo "BUERGER_DB_PATH=/tmp/test_buerger_db.json" >> $GITHUB_ENV
```

CI-Workflow für eVote



Abhängigkeiten installieren & Linting mit Ruff (Code-Qualität)

- Zuerst wird pip aktualisiert
- Alle Pakete definierten in Requirements.txt werden installiert
- `ruff check . --fix`: Analysiert den Code (Python-Dateien) auf Stil- und einfache Fehler und versucht Sie automatisch zu reparieren.
- `continue-on-error: true`: Auch wenn Ruff Probleme findet, bricht die CI nicht ab.

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    if [ -f requirements.txt ]; then
      pip install -r requirements.txt
    else
      python -m pip install fastapi "uvicorn[standard]" jinja2 httpx
      python -m pip install "pydantic[email]" email-validator
      python -m pip install pytest ruff pytest-cov
    fi

- name: Lint (ruff)
  run: ruff check . --fix
  continue-on-error: true
```

CI-Workflow für eVote



Statische Verzeichnisse & Debug

- Erstellt das Verzeichnis apps/static, falls es noch nicht existiert.
- Verhindert Fehler, falls Teile deiner App erwarten, dass dieses Verzeichnis vorhanden ist (z. B. für CSS, Bilder).
- Ausgabe von: wichtigen Umgebungsvariablen, Python-Version und installierte Pakete

```
- name: Ensure static dirs exist
```

```
run: mkdir -p apps/static
```

```
- name: Debug environment
```

```
run: |
```

```
  echo "=== Environment Variables ==="
```

```
  echo "PYTHONPATH=$PYTHONPATH"
```

```
  echo "TESTING=$TESTING"
```

```
  echo "BUERGER_DB_PATH=$BUERGER_DB_PATH"
```

```
  echo ""
```

```
  echo "=== Test Database Status ==="
```

```
  ls -la /tmp/test_buerger_db.json || echo "❌ Test DB not found"
```

```
  echo ""
```

CI-Workflow für eVote



Test-Kollektion prüfen & Tests mit Coverage ausführen

- `pytest --collect-only`:
Pytest sucht nur nach Tests und listet sie auf – er führt sie noch nicht aus.
- Frühe Erkennung von Problemen wie:
Syntaxfehler in Testdateien, falsche Testbenennung, Importprobleme
- Führt alle Tests aus (pytest).

```
- name: Check test collection
  run: |
    echo "=== Attempting to collect tests ==="
    python -m pytest --collect-only -vv
  continue-on-error: true

- name: Run tests with coverage
  run: |
    python -m pytest -v \
      --capture=no \
      --disable-warnings \
      --cov=apps \
      --cov-report=xml \
      --cov-report=term \
      --tb=long \
      ....
```

CI-Workflow für eVote



Coverage-Report & SonarQube Cloud-Analyse

- Führt eine Code-Qualitäts-Analyse mit SonarQube Cloud aus:
- Lädt die Datei coverage.xml als Build-Artefakt hoch.
- `if: always()`:
Der Scan läuft immer, auch wenn Tests fehlschlagen.

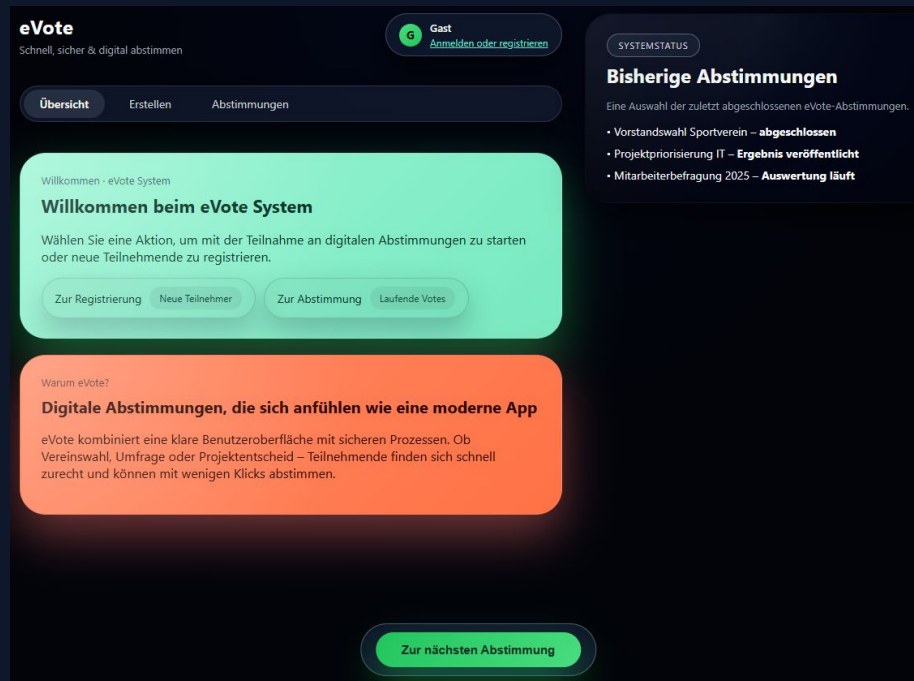
```
- name: Upload coverage artifact
  if: success()
  uses: actions/upload-artifact@b4b15b8c7c6ac21ea08fcf65892d2ee8f75cf882
  with:
    name: coverage
    path: coverage.xml

- name: SonarCloud Scan
  if: always()
  uses: SonarSource/sonarcloud-github-action@master
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Zusammenfassung & Ausblick (1)

Bisheriger Projektfortschritt

- **Git-Workflow** etabliert mit Branch-Strategie und Pull Request Reviews
- **Bounded Contexts** modelliert (DDD)
- **Event-basierte Kommunikation** skizziert
- **TDD-Ansatz** für Bürgerverwaltung implementiert
- **CI/CD-Pipeline** funktionsfähig mit automatisierten Tests
- **SonarCloud-Integration** für kontinuierliche Code-Qualitätsüberwachung
- **Pydantic-Validierung** für alle Aggregate Roots
- Erstes **Frontend** (FastAPI)



Zusammenfassung & Ausblick (2)

Kritische Reflektion

- **Steile Lernkurve:** Wenig Vorkenntnisse – DDD, TDD, CI/CD, Pydantic, pytest waren größtenteils neu
- **Teamarbeit als Schlüssel:** Intensive Zusammenarbeit, Code Reviews, Pair-Programming und LLM-Einsatz bewältigten Herausforderungen
- **Sichtbare Fortschritte:** Regelmäßige Abstimmungen und gemeinsames Troubleshooting ermöglichten schnellen Wissensaufbau

Ausblick in die Zukunft

- Das “**Fundament**” wurde gebaut: Alles, was hinter den Kulissen wichtig ist (Tests, automatische Prüfungen, Codequalität, klare Strukturen)
- Jetzt geht's darum, dass der „**Bau**“ auch von außen (über das Frontend) nutzbar und sichtbar wird – die Nutzeroberfläche bekommt Verbindung zu den Funktionen im Hintergrund (mittels API)

Vielen Dank!

Fragen?